

SE & Cloud Computing course

04 - Science “vs” Engineering, and ML Engineering (SE4ML)

Robert Feldt

Division of Software Engineering , Dept of Computer Science and Engineering
Chalmers University of Technology



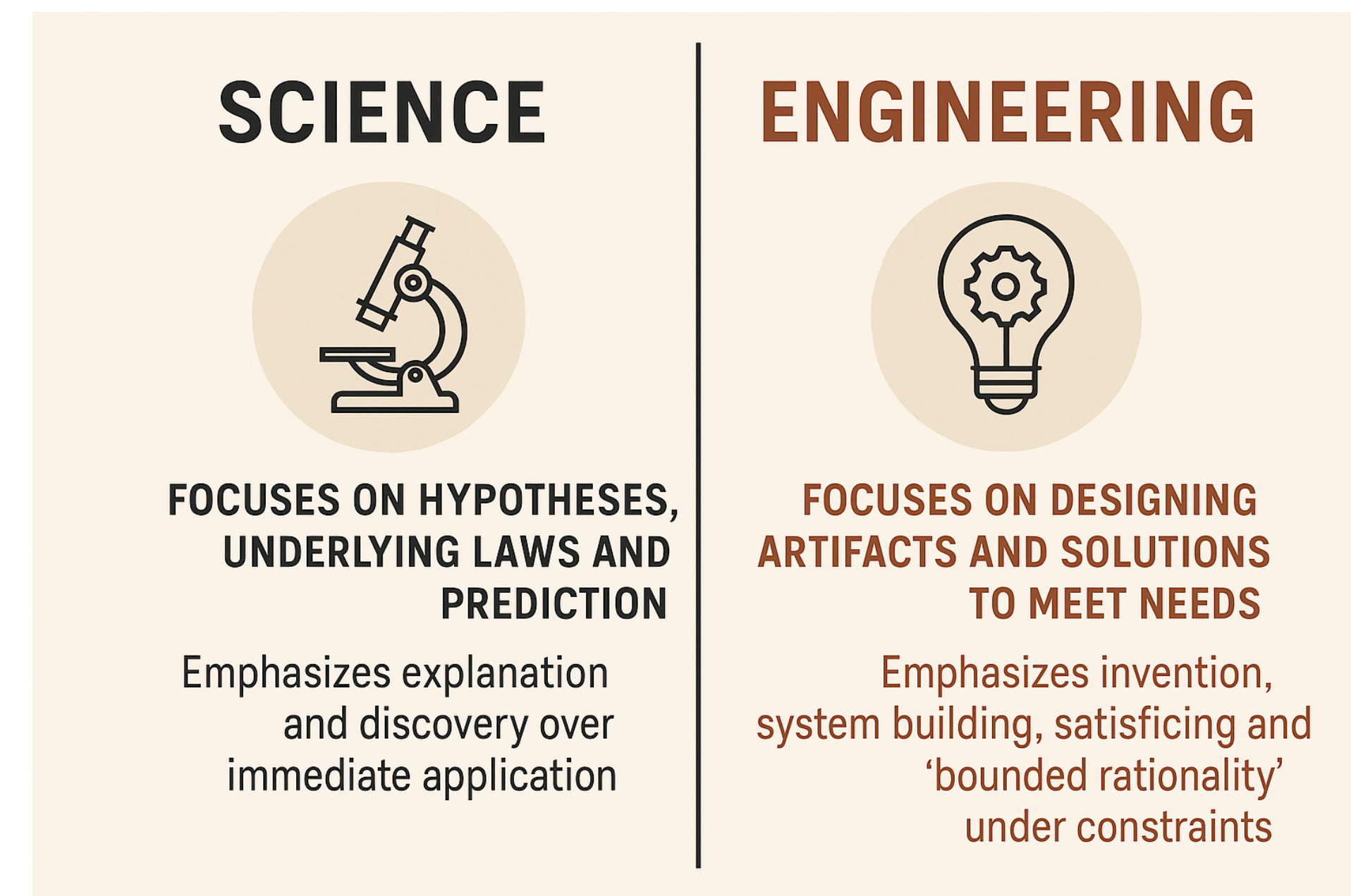
robert.feldt@chalmers.se



@drfeldt

Science “vs” Engineering

- **Science** (Predictive/Discovery)
 - Scientists ask “why does this work?”
- **Engineering** (Constructive/Solution-driven)
 - Engineers ask “how can we make it work reliably?”
 - Involves “epistemic diversity” (design & historical context) – making it distinct from pure science
- **ML as Hybrid??**
 - Modern ML sits between science and engineering – use statistical hypotheses and methods but must also build end-to-end systems.
 - ML as both an (empirical) science and a design discipline
 - AI/ML teams effectively create their own **science-engineering loops**: they run experiments, push models to production, observe failures or new data, and then refine (an iterative flow of experiment→deploy→refine).



Technology readiness levels for machine learning systems

Received: 21 December 2020

Accepted: 2 September 2022

Published online: 20 October 2022

 Check for updates

Alexander Lavin ^{1,2}✉, Ciarán M. Gilligan-Lee ^{3,4}, Alessya Visnjic ⁵,
Siddha Ganju ^{2,6}, Dava Newman ⁷, Sujoy Ganguly⁸, Danny Lange⁸,
Atılım Güneş Baydin ⁹, Amit Sharma ¹⁰, Adam Gibson¹¹, Stephan Zheng ¹²,
Eric P. Xing^{13,14}, Chris Mattmann ¹⁵, James Parr² & Yarin Gal ¹⁶

The development and deployment of machine learning systems can be executed easily with modern tools, but the process is typically rushed and means-to-an-end. Lack of diligence can lead to technical debt, scope creep and misaligned objectives, model misuse and failures, and expensive consequences. Engineering systems, on the other hand, follow well-defined processes and testing standards to streamline development for high-quality, reliable results. The extreme is spacecraft systems, with mission critical measures and robustness throughout the process. Drawing on experience in both spacecraft engineering and machine learning (research through product across domain areas), we've developed a proven systems engineering approach for machine learning and artificial intelligence: the *Machine Learning Technology Readiness Levels* framework defines a principled process to ensure robust, reliable, and responsible systems while being streamlined for machine learning workflows, including key distinctions from traditional software engineering, and a lingua franca for people across teams and organizations to work collaboratively on machine learning and artificial intelligence technologies. Here we describe the framework and elucidate with use-cases from physics research to computer vision apps to medical diagnostics.

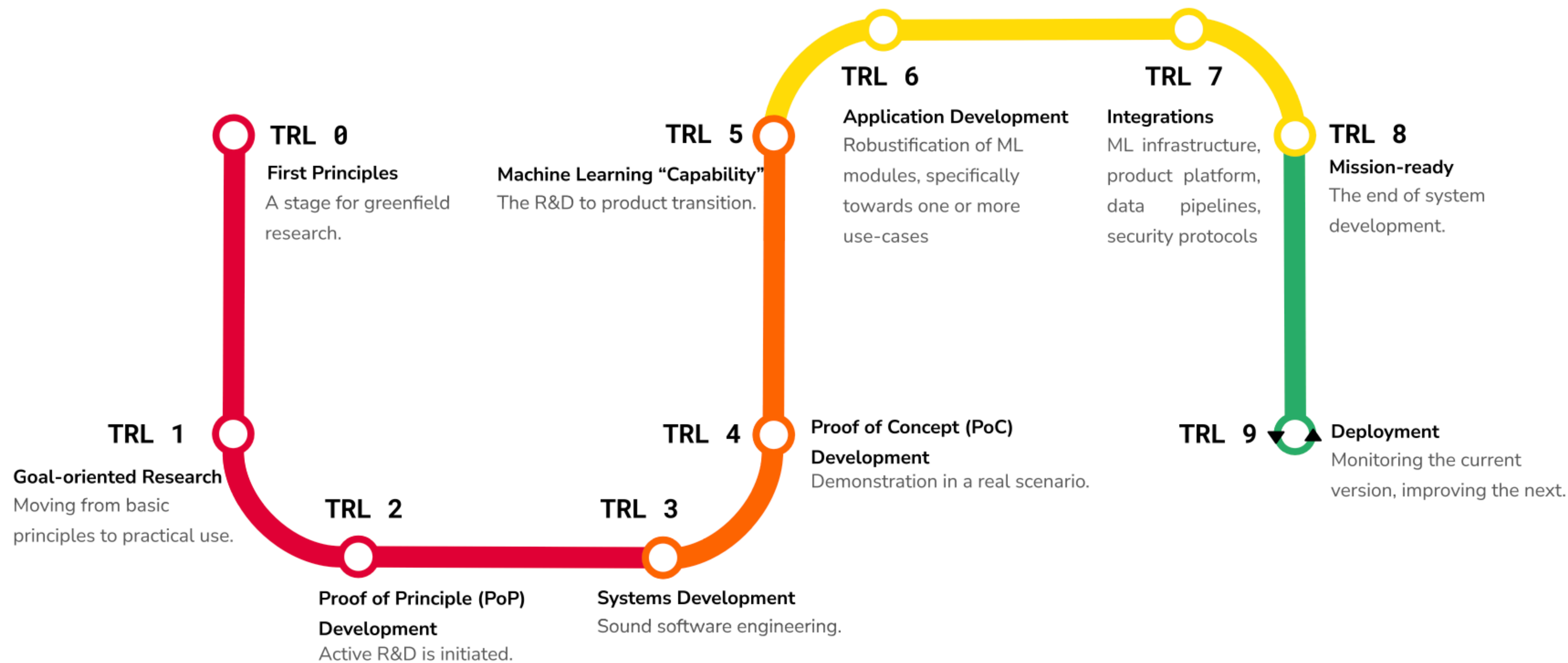


Fig. 1 | MLTRL spans research (red) through prototyping (orange), production (yellow), and deployment (green). Most ML workflows prescribe an isolated, linear process of data processing, training, testing, and serving a model³⁷. Those workflows fail to define how ML development must iterate over that basic

process to become more mature and robust, and how to integrate with a much larger system of software, hardware, data, and people. Not to mention MLTRL continues beyond deployment: monitoring and feedback cycles are important for continuous reliability and improvement over the product lifetime.

ML-TRL levels

- **0. First Principles:**
 - Theoretical ideation: explore foundational concepts, mathematical formulation
- **1. Goal-oriented research:**
 - Early experimentation: using sample/synthetic data to test specific properties of algorithms
- **2. Proof of Principle:**
 - Deeper R&D using testbeds or simulations; formalize research requirements and V&V steps
- **3. System development:**
 - Develop prototype code focused on software quality, modularity, and testability
- **4. Proof of Concept:**
 - Validate technology in real or representative environments using real-world data
- **5. ML Capability:**
 - Transition from isolated model to integrated system module; x-functional collaboration begins

ML-TRL levels

- **6. Application Development:**
 - Production-level software engineering; consider deployment constraints, explainability, and compliance
- **7. Integrations**
 - Integrate ML into larger systems; perform rigorous testing (e.g., metamorphic, intervention)
- **8. Mission-ready**
 - Final testing (A/B, shadow) & validation for deployment under real-world constraints
- **9. Deployment**
 - Live system monitoring and maintenance engineering; track drift, logging, retraining needs

ML: Mutual Research-Engineering Loops

- New research quickly leads to new tools and engineering breakthroughs
 - Self-attention/Transformers -> BERT/GPT-X -> Translation/Chatbots...
- Practical problems drive new theory/science
 - Adversarial attacks in vision systems spurred work on DNN robustness
 - Privacy needs in products -> Federated learning & Differential privacy
 - Engineering and cost prediction needs lead to scaling law hypotheses!?

Software Engineering for Machine Learning: A Case Study

Saleema Amershi	Andrew Begel	Christian Bird	Robert DeLine	Harald Gall
<i>Microsoft Research</i>	<i>Microsoft Research</i>	<i>Microsoft Research</i>	<i>Microsoft Research</i>	<i>University of Zurich</i>
Redmond, WA USA	Redmond, WA USA	Redmond, WA USA	Redmond, WA USA	Zurich, Switzerland
samershi@microsoft.com	andrew.begel@microsoft.com	cbird@microsoft.com	rdeline@microsoft.com	gall@ifi.uzh.ch

Ece Kamar	Nachiappan Nagappan	Besmira Nushi	Thomas Zimmermann
<i>Microsoft Research</i>	<i>Microsoft Research</i>	<i>Microsoft Research</i>	<i>Microsoft Research</i>
Redmond, WA USA	Redmond, WA USA	Redmond, WA USA	Redmond, WA USA
eckamar@microsoft.com	nachin@microsoft.com	besmira.nushi@microsoft.com	tzimmer@microsoft.com

Abstract—Recent advances in machine learning have stimulated widespread interest within the Information Technology sector on integrating AI capabilities into software and services. This goal has forced organizations to evolve their development processes. We report on a study that we conducted on observing software teams at Microsoft as they develop AI-based applications. We consider a nine-stage workflow process informed by prior experiences developing AI applications (e.g., search and NLP) and data science tools (e.g. application diagnostics and bug reporting). We found that various Microsoft teams have united this workflow into preexisting, well-evolved, Agile-like software engineering processes, providing insights about several essential engineering challenges that organizations may face in creating large-scale AI solutions for the marketplace. We collected some best practices from Microsoft teams to address these challenges. In addition, we have identified three aspects of the AI domain that make it fundamentally different from prior software application domains: 1) discovering, managing, and versioning the data needed for machine learning applications is much more complex and difficult than other types of software engineering, 2) model customization and model reuse require very different skills than are typically found in software teams, and 3) AI components are more difficult to handle as distinct modules than traditional software components — models may be “entangled” in complex ways and experience non-monotonic error behavior. We believe that the lessons learned by Microsoft teams will be valuable to other organizations.

Index Terms—AI, Software engineering, process, data

techniques that have powered recent excitement in the software and services marketplace. Microsoft product teams have used machine learning to create application suites such as Bing Search or the Cortana virtual assistant, as well as platforms such as Microsoft Translator for real-time translation of text, voice, and video, Cognitive Services for vision, speech, and language understanding for building interactive, conversational agents, and the Azure AI platform to enable customers to build their own machine learning applications [1]. To create these software products, Microsoft has leveraged its preexisting capabilities in AI and developed new areas of expertise across the company.

In this paper, we describe a study in which we learned how various Microsoft software teams build software applications with customer-focused AI features. For that, Microsoft has integrated existing Agile software engineering processes with AI-specific workflows informed by prior experiences in developing early AI and data science applications. In our study, we asked Microsoft employees about how they worked through the growing challenges of daily software development specific to AI, as well as the larger, more essential issues inherent in the development of large-scale AI infrastructure and applications. With teams across the company having differing amounts of work experience in AI, we observed that many issues reported



Software Engineering for Machine Learning

Amershi et al., ICSE-SEIP 2019

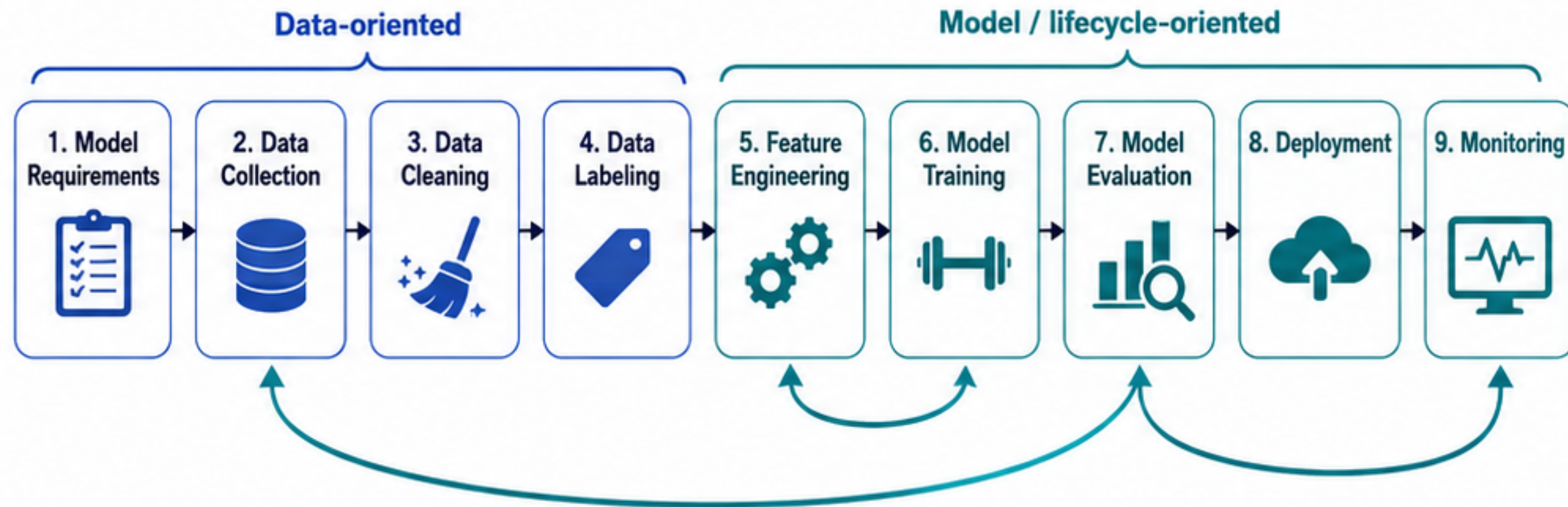


Core idea: ML changes software engineering because teams must engineer not only code, but also data, models, pipelines, evaluation, deployment, and monitoring.

STUDY SNAPSHOT

- Microsoft industrial case study
- 14 interviews
- 551 survey responses
- 9-stage ML workflow
- Focus: real product teams building ML-enabled systems
- Main question:** How must software engineering adapt for ML systems?

NINE-STAGE ML WORKFLOW



ML development is iterative, experimental, and feedback-heavy.

ENGINEERING IMPLICATIONS: WHAT TEAMS SHOULD DO

- A. Expand SE beyond code**
Treat data, models, and pipelines as first-class engineering artifacts.
- B. Adapt Agile / DevOps**
Use a data-centered workflow with experimentation and rapid feedback loops.
- C. Automate rigorously**
Automate pipelines, tests, training, deployment, and reproducibility where possible.
- D. Evaluate and monitor continuously**
Model quality is not a final step: evaluation and monitoring continue after deployment.



Process maturity = documented, repeatable, automated, measured, improved.

WHAT MAKES ML DIFFERENT?



1. Data is harder than code
Finding, managing, and versioning data is difficult.
Data quality strongly shapes system behavior.



2. Reuse needs ML expertise
Customizing or reusing models is not plug-and-play.
Teams need stronger ML skills than in ordinary software projects.



3. Models are entangled
ML components interact in non-obvious ways.
Fixing one part can hurt overall system quality (*non-monotonic errors*).



Key takeaway: ML-enabled software engineering is ordinary software engineering expanded around data, models, pipelines, monitoring, and iterative feedback — not code-centric development alone.

Still just personal assistants? – A multiple case study of generative AI adoption in software organizations



Kai-Kristian Kemell ^a,* , Matti Saarikallio ^b, Anh Nguyen-Duc ^c, Pekka Abrahamsson ^a

^a Tampere University, Department of Information Technology and Computer Science, Tampere, 33014, Finland

^b Independent researcher, Finland

^c University of South-Eastern Norway, Bø I Telemark, 3800, Norway

ARTICLE INFO

Keywords:

Large language model
Software engineering
Case study
Generative AI
Artificial intelligence
Technology adoption
Empirical research

ABSTRACT

Context: Generative AI (GenAI) is argued to transform software engineering (SE) in various ways, and GenAI tools show promise for various SE tasks. Software organizations across the globe are currently exploring the use of GenAI for SE.

Objective: While numerous studies have recently been published on GenAI, few studies have looked at the adoption of these tools and their usage from an organizational point of view, focusing instead on individual users. Our objective is to understand how organizations adopt these tools and what their impacts are in industrial contexts, with a focus on the European perspective.

Method: We conducted a multiple case study of seven European companies. We collected data through semi-structured interviews (n=15), as well as through longitudinal observation in one case company. All data were analyzed using thematic analysis.

Results: We analyzed 28 transcripts, resulting in 456 quotations and 557 code occurrences split between 66 individual codes that were categorized under 6 high-level themes. We identified 25 types of tasks GenAI was currently being used for in our case organizations. We identified 12 benefits for GenAI in SE and 10 adoption and use challenges. Key adoption challenges for organizations include data privacy and legislative concerns, the emerging and fast-moving market of GenAI tools, difficulty of measuring the positive impact of the tools, and potential change resistance. For individuals, the key challenges are related to prompting, such as understanding what a good prompt is, and how to write prompts for specific tasks.

Conclusion: GenAI adoption is becoming widespread in SE, but good practices and use cases are still emerging. While GenAI can potentially produce various benefits in SE, companies and individual users are facing various challenges in making the most of GenAI in SE. Overall, GenAI is still primarily used as a personal assistant.

Generative AI Adoption in Software Organizations

Kemell et al., Information and Software Technology 2025

Still just personal assistants?

STUDY SNAPSHOT

 7 European companies

 15 semi-structured interviews

 1 longitudinal observation case

 28 transcripts analyzed

 456 quotations / 557 code occurrences

 25 task types, 12 benefits
10 challenges identified

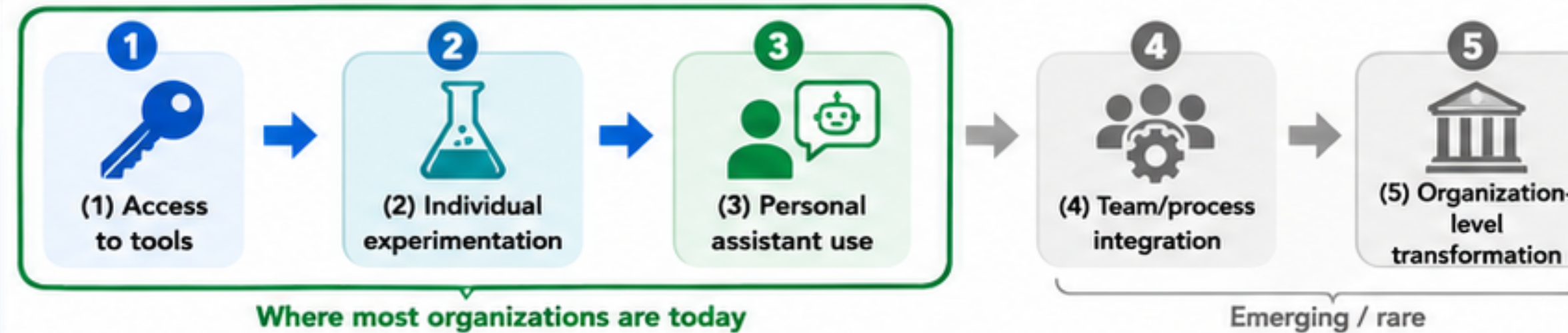
Research focus

How are software organizations adopting GenAI, what benefits and challenges do they see, and how is SE changing?



Core idea: GenAI is already widely used in software organizations, but mostly as an individual personal assistant. Organizational integration, governance, measurement, and process transformation are still immature.

CURRENT ADOPTION PATTERN



Observed reality: widespread use, but limited workflow transformation.



None of the case organizations used LLMs in ways that notably transformed SE on an organizational level.

WHAT GENAI IS USED FOR

25 task types identified

A. Coding help

- code generation
- refactoring
- code migration
- code review
- explain code

B. Testing & debugging

- test generation
- debugging
- error handling
- updating deprecated code

C. Docs & requirements

- documentation/ comments
- acceptance criteria
- requirements engineering
- project plans
- templates

D. Information & analysis

- information retrieval
- summarization
- data processing/ analysis
- translation

E. Communication & business support

- meeting notes/ transcription
- customer support
- marketing text
- CV creation
- product content

Use cases are broad, but mostly assistant-like rather than deeply integrated automation.

SO WHAT DOES THIS MEAN FOR SE?



A. Adoption is socio-technical

governance, clients, and company policy matter.



B. Prompting is a real skill

useful results are not automatic.



C. Measurement is weak

companies want evidence but struggle to quantify value.



D. Transformation is still emerging

GenAI changes habits faster than organizational processes.

PERCEIVED BENEFITS



Saves time and increases perceived productivity



Removes repetitive or menial work



Helps developers stay in flow



Supports learning, explanation, and information retrieval



Helps with ideation and drafting



Important: benefits were self-reported; organizations could not quantify ROI.

ADOPTION CHALLENGES

Organization



Data privacy & legislative concerns



Constant emergence of new tools / tool churn



Black-box models



Hard to measure impact / ROI



Bureaucracy and change resistance

Individual user



Prompting is hard to learn



Prompts are difficult to evaluate



Tool usability issues



Erroneous outputs / hallucinations



Hard to use for complex programming problems



Interpret with care

- Benefits were not directly measured.
- Most cases had limited respondent coverage.
- All companies were European.



Key takeaway: The main bottleneck is not a lack of possible GenAI use cases in software engineering, but turning widespread personal use into governed, measurable, process-level organizational change.



A 2030 Roadmap for Software Engineering

MAURO PEZZÈ, Università della Svizzera italiana, Lugano, Switzerland, Università degli Studi di Milano-Bicocca, Milan, Italy, and Constructor University, Schaffhausen, Switzerland

SILVIA ABRAHÃO, Universitat Politècnica de Valencia, Valencia, Spain

BIRGIT PENZENSTADLER, Chalmers University of Technology, Gothenburg, Sweden

DENYS POSHYVANYK, William & Mary, Williamsburg, VA, USA

ABHIK ROYCHOUDHURY, National University of Singapore, Singapore, Singapore

TAO YUE, Beihang University, Beijing, China

The landscape of software engineering has dramatically changed in recent years. The impressive advances of artificial intelligence are just the latest and most disruptive innovation that has remarkably changed the software engineering research and practice. This special issue shares a roadmap to guide the software engineering community in this confused era. This roadmap is the outcome of a 2-day intensive discussion at the *2030 Software Engineering* workshop. The roadmap spotlights and discusses seven main landmarks in the new software engineering landscape: artificial intelligence for software engineering, human aspects of software engineering, software security, verification and validation, sustainable software engineering, automatic programming, and quantum software engineering. This editorial summarizes the core aspects discussed in the 37 papers that comprise the seven sections of the special issue and guides the interested readers throughout the issue. This roadmap is a living body that we will refine with follow-up workshops that will update the roadmap for a series of forthcoming ACM TOSEM special issues.



Fig. 2. Overview of McLuhan's tetrad.

Applies McLuhan Tetrad to 7 key disruptors

- 1. Artificial Intelligence for Software Engineering
- 2. Software Engineering by and for Humans
- 3. Sustainable Software Engineering
- 4. Automatic Programming
- 5. Security and Software Engineering
- 6. Verification and Validation
- 7. Quantum Software Engineering

AI for SE

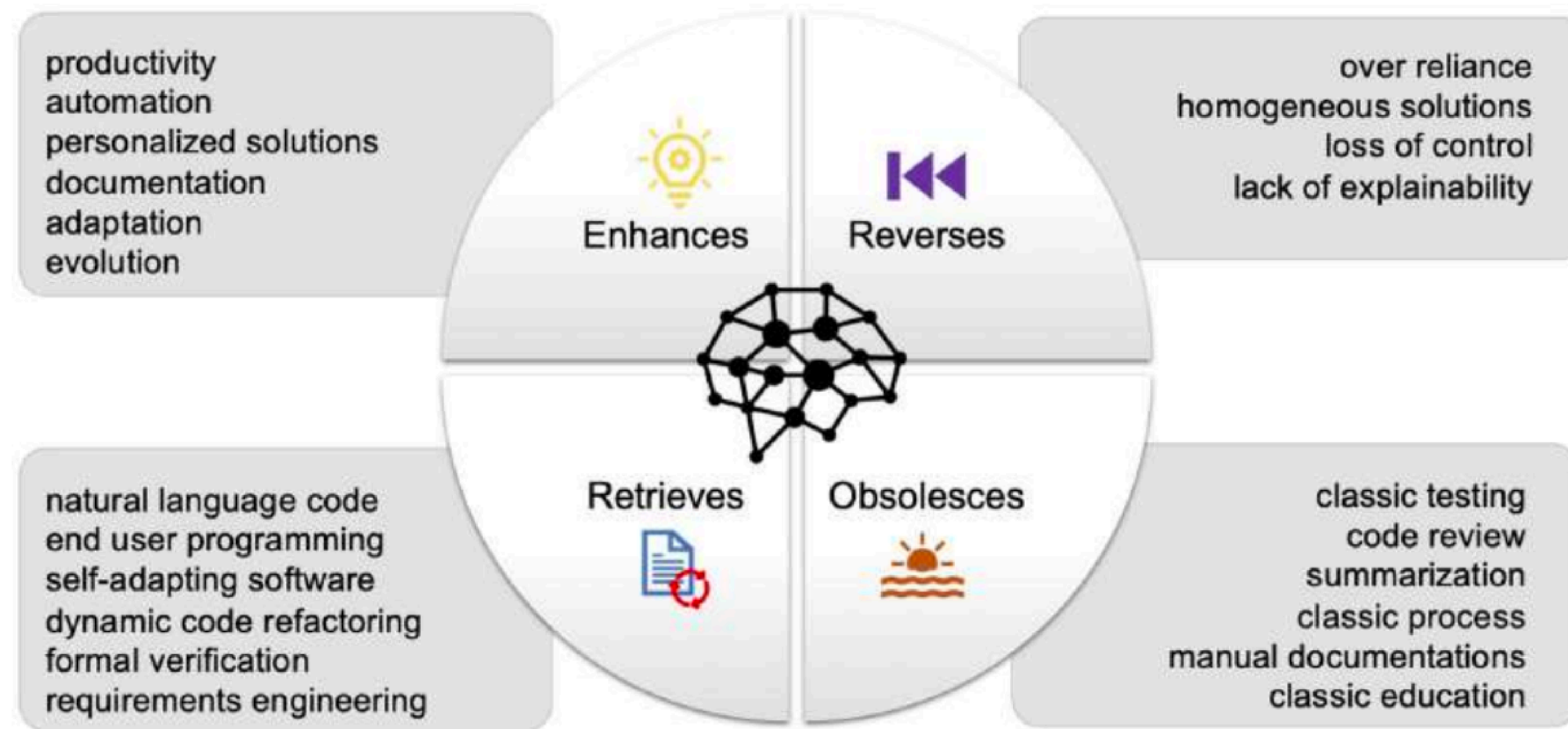


Fig. 3. The disruptive impact of artificial intelligence on software engineering.

- Enhances: productivity and lifecycle automation
 - AI is not just a better IDE autocomplete; it expands across the SDLC
- Obsolesces: “coding-first” SE practice
 - from implementation labor to specification, supervision, and assurance
- Reverses: overreliance, opacity, and loss of control
 - AI can hide computational decisions, weaken explainability, and can push human creativity/judgment out

SE for Humans

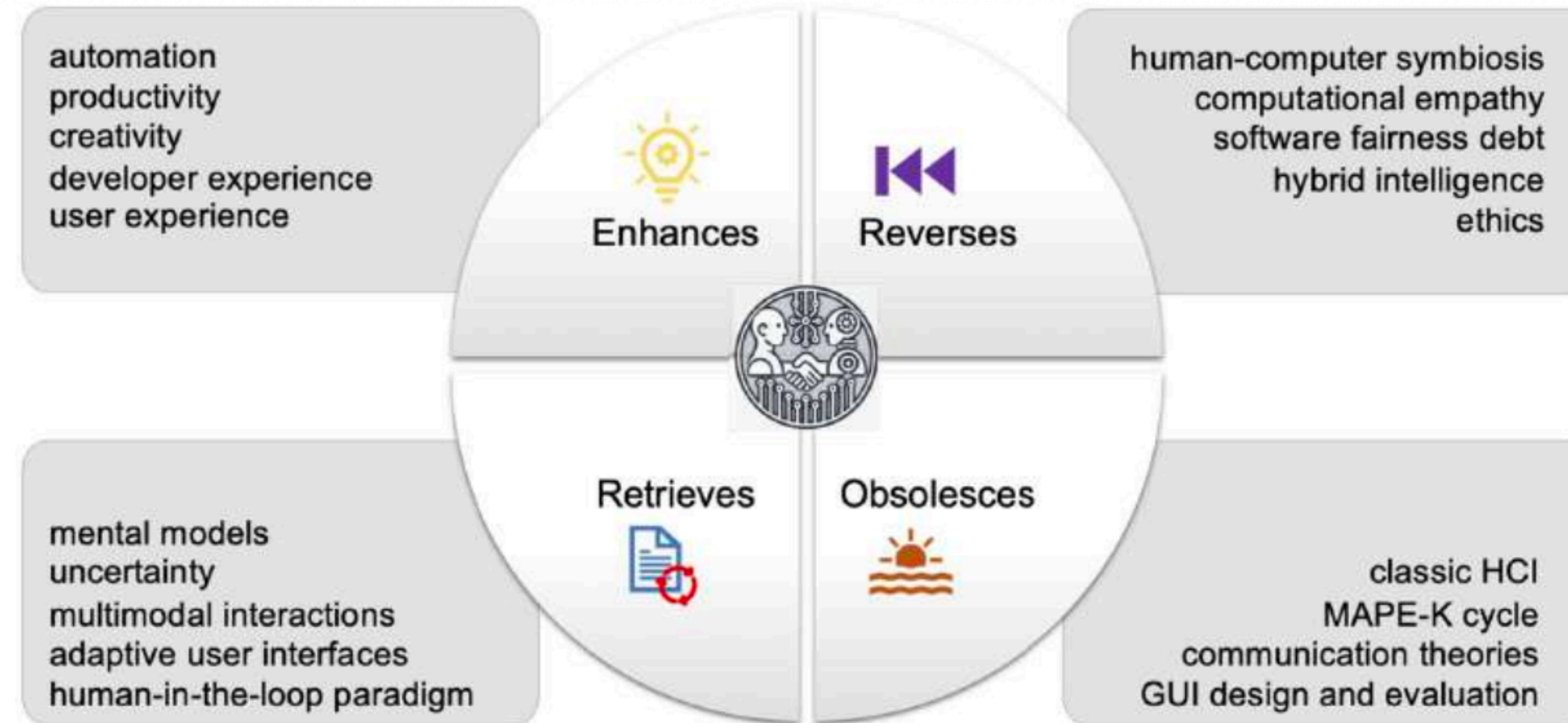


Fig. 4. The disruptive impact of human-artificial intelligence interaction in software engineering.

- Enhances: hybrid human-AI collaboration
 - AI agents become teammates in coding, QA, and decision support
- Obsolesces: humans as only “users”
 - Humans become active parts of software ecosystems, not just external users of finished systems
- Reverses: deskilling
 - can become dependency, loss of expertise, reduced agency

V&V



Fig. 8. The disruptive impact of artificial intelligence on software verification and validation.

- Enhances: auto-generate test oracles and metamorphic relations
 - Molina et al on generating test oracles
- Retrieves: Field-testing and runtime monitoring
 - Richer ways to detect and analyse bugs, also “online”
- Reverses into loss of trust
 - “Verifying the verifier”!? Who ensures the Agentic tester is “correct”

Final synthesis: Towards an AI Engineering Manifesto

